While Loops In Java

Generated Aug 13, 2026 8:04 AM

What We'll Learn

By the end of this lesson, students can:

- Explain how a `while` loop repeats statements.
- Identify the loop condition and loop body.
- Create a counter-controlled `while` loop.
- Use a `while` loop with user input.
- Prevent common problems such as infinite loops.

Before We Start

Imagine that you need to display the same message five times.

You could write:

```
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
System.out.println("Welcome to Java!");
```

This works, but repeating the same statement makes the program longer and more difficult to update.

How can we tell Java to repeat a statement without writing it several times?

Let's Understand

A loop allows a program to repeat a group of statements.

A `while` loop repeatedly executes its statements while a condition remains `true`.

Basic Syntax

```
while (condition) {
    // Statements to repeat
}
```

The condition is checked before the statements inside the loop are executed.

How A While Loop Works

1. Java checks the condition.
2. If the condition is `true`, the loop body executes.
3. The program returns to the condition.
4. The condition is checked again.
5. The loop continues while the condition is `true`.
6. The loop stops when the condition becomes `false`.

Creating A Counter-Controlled Loop

A counter-controlled loop uses a variable to track the number of repetitions.

```
int count = 1;

while (count <= 5) {
    System.out.println(count);
    count++;
}
```

Output:

```
1
2
3
4
5
```

This loop has three important parts:

- Initialization: `int count = 1;`
- Condition: `count <= 5`
- Update: `count++;`

Tracing The Loop

Value of count	Is count <= 5?	Result
1	true	Display 1
2	true	Display 2
3	true	Display 3
4	true	Display 4
5	true	Display 5
6	false	Stop the loop

Infinite Loops

An infinite loop occurs when the condition never becomes `false`.

```
int count = 1;

while (count <= 5) {
    System.out.println(count);
}
```

The value of `count` never changes, so the condition `count <= 5` always remains `true`.

To prevent this problem, update the loop variable inside the loop.

```
count++;
```

A While Loop May Not Execute

Because the condition is checked before the loop body, a `while` loop may execute zero times.

```
int number = 10;

while (number < 5) {
    System.out.println(number);
}
```

The condition is already `false`, so the statement inside the loop does not execute.

Let's Practice

Practice 1: Trace The Program

Study the following program:

```
public class LoopTracing {
    public static void main(String[] args) {
        int number = 2;

        while (number <= 8) {
            System.out.println(number);
            number = number + 2;
        }
    }
}
```

Answer the following questions:

1. What is the initial value of `number`?
2. What condition controls the loop?
3. How does the value of `number` change?
4. What numbers will the program display?
5. What is the value of `number` when the loop stops?

Practice 2: Complete The Loop

Complete the missing parts so the program displays the numbers from `1` to `5`.

```
public class CompleteLoop {
    public static void main(String[] args) {
        int count = ______;

        while (count ______ 5) {
            System.out.println(count);
            count______;
        }
    }
}
```

Practice 3: Find And Correct The Error

The following program is supposed to display the numbers from `5` down to `1`.

```
public class FixTheLoop {
    public static void main(String[] args) {
        int number = 5;

        while (number >= 1) {
            System.out.println(number);
```

```
            number++;
        }
    }
}
```

Identify the error and rewrite the incorrect statement.

Practice 4: Create A While Loop

Create a Java program that:

1. Starts with the number `10`.
2. Uses a `while` loop.
3. Displays the even numbers from `10` to `20`.
4. Displays one number on each line.
5. Stops after displaying `20`.

Resources / References

- Java Development Kit
- Visual Studio Code
- Java `while` loop documentation
- Java `Scanner` class